Tools for software development

- Lab 6 - **API Testing with Postman and Project Finalization**

POSTMAN

doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk

ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Where we are in the course

The project moves from setup to professional test documentation.

Today: build an API test collection, export evidence, finalize the report and presentation.

Lab 6 goals

By the end of this lab, each team should have API tests and defense materials

API testing skills

- Create a Postman collection
- Send GET and POST requests
- Check status codes and response fields
- Create positive and negative API tests
- Run the whole collection

Expected team output

- Finalize final project report
- Prepare defense presentation
- Confirm repository readiness

What is an API?

Application Programming Interface

Concept

An API allows software systems to communicate with each other. It exposes operations and data without requiring users to click through the graphical interface.

```
Web application  →  API  →  Backend service
Mobile app       →  API  →  Server
Testing tool     →  API  →  Response validation
```

Testing perspective

API testing verifies whether the backend responds correctly to requests: status code, returned data, errors and business rules.

Where API testing fits

Testing under the UI layer

Postman tests this communication directly.

Why API testing matters

Fast, stable and close to business logic

- Tests backend logic directly, without UI dependencies
- Runs faster than browser-based UI tests
- Is less fragile than Selenium tests affected by locators or layout changes
- Supports positive and negative scenarios
- Can be automated in CI/CD pipelines using tools such as Newman
- Helps find defects before they appear in the user interface

REST API basics

Most public APIs use HTTP methods

Method	Meaning	Example
GET	Read data	Get list of users
POST	Create data	Create new post/user
PUT	Update full object	Replace user profile
PATCH	Update part of object	Change one field
DELETE	Delete object	Delete resource by ID

HTTP status codes

Status code is the first API checkpoint

Code	Meaning	Typical situation
200	OK	Successful GET
201	Created	Successful POST
400	Bad Request	Invalid input
401	Unauthorized	Missing or wrong authentication
403	Forbidden	No permission
404	Not Found	Resource does not exist
500	Server Error	Backend failure

Anatomy of an API request

Method + URL + headers + body

```
Method:   GET / POST / PUT / DELETE
URL:      endpoint address
Headers:  metadata, authorization, content type
Body:     data sent to server
Response: status code + response body
```

Example

GET https://jsonplaceholder.typicode.com/users/1

Expected: status code 200 and response contains id, name and email.

Testing question

What exactly do we expect from this request, and how can we prove it automatically?

JSON response

API data are often returned as JSON

```
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz"
}
```

- Does the field exist?
- Does it have the expected value?
- Does it have the expected data type?
- Does the response follow the expected structure?
- Is an error returned for invalid input?

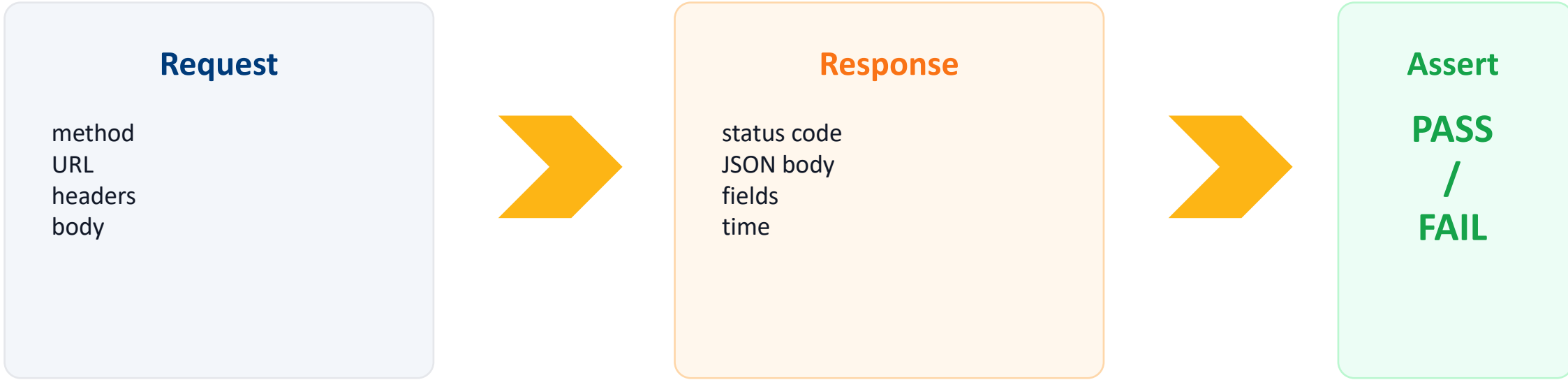
What should we test in an API?

API tests should verify behavior, not just availability

- Status code: 200, 201, 400, 401, 404...
- Required response fields and JSON structure
- Business rules and validation rules
- Positive scenarios with valid input
- Negative scenarios with invalid input or missing data
- Authentication and authorization when available
- Response time as a basic smoke check

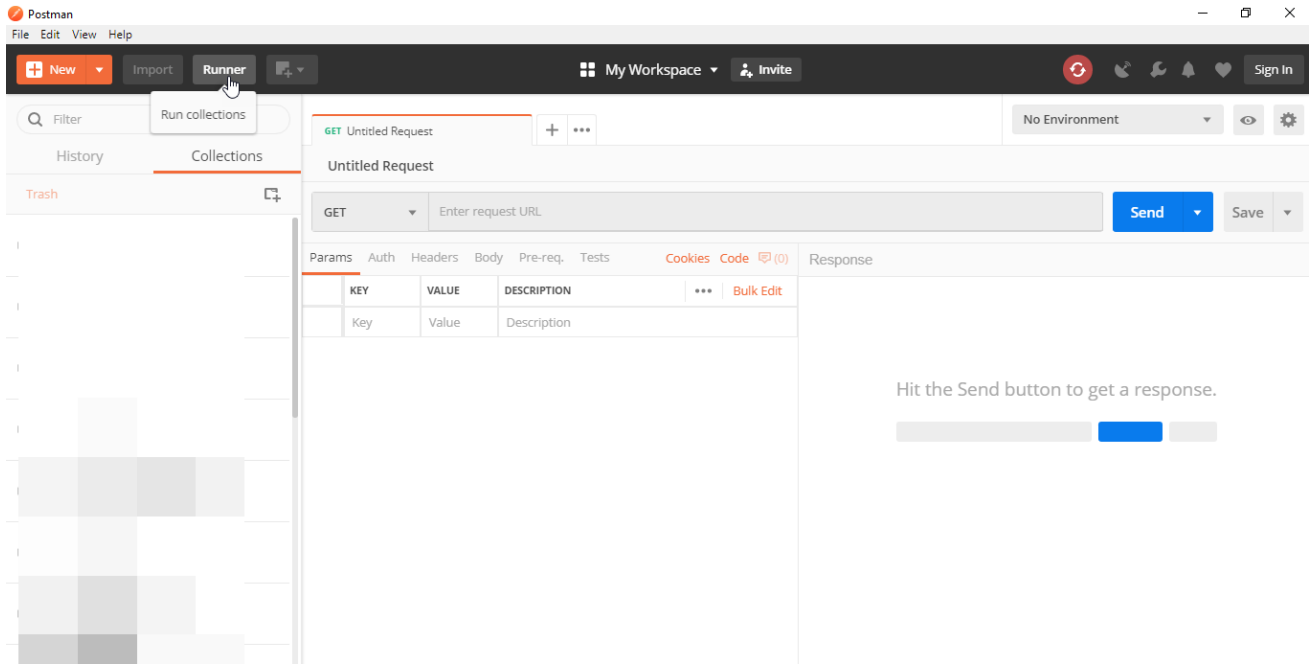
API testing is not just opening a URL

A test verifies behavior, not only availability



Postman

- scalable API testing tool
- we can incorporate it into CI/CD techniques
- developed since 2012 to simplify API tests



Why choose Postman ?

- **accessibility** *
  own account and online synchronization
- **using collections**
  creation of collections for individual test sets
- **cooperation** *
  import/export, file sharing, direct link
- **creating sets**
  the set can be used in different collections for the same test
- **test creation**
  test checkpoints – response to HTTP request – clarity
- **test automation**
  collection runner , Newman cmd
- **debugging**
  console for debugging assistance
- **CI – “continuous integration”**

Types of API tests in this lab

Use a small but meaningful test set

Type	Example	What it proves
Positive	GET existing user returns 200	Happy path works
Negative	GET non-existing user returns 404	Error handling works
Validation	POST missing required field	Input rules are enforced
Schema	Response contains id, name, email	Contract is stable
Performance smoke	Response under 1000 ms	Basic responsiveness

Choose an API for your team project

Use your own API if available; otherwise use a public sandbox

Own project API

- Best option if stable
- Must be accessible to the teacher
- Needs clear endpoints
- Must not require sensitive data

Public API sandbox

- JSONPlaceholder - https://jsonplaceholder.typicode.com/
- Reqres
- Restful Booker
- DummyJSON
- Fake Store API

Exercise 1: Create a Postman collection

The collection is the container for all API tests

```
Collection name:
TSD 2026 - Team Name - API Tests

Export later to:
automation/postman/team-name-api-
tests.postman_collection.json
```

Collection should contain

at least 4 requests
GET and POST examples
positive and negative cases
Postman test assertions

Use folders in Postman if your API has more domains, e.g. users, products, carts.

Exercise 2: GET list of users

First API smoke test

```
GET https://jsonplaceholder.typicode.com/users
```

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Response contains users", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.length).to.be.above(0);
});

pm.test("First user has email", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData[0]).to.have.property("email");
});
```

Exercise 3: GET user by ID

Check fields and exact values

```
GET https://jsonplaceholder.typicode.com/users/1
```

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});

pm.test("User ID is 1", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.id).to.eql(1);
});

pm.test("User has required fields", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property("name");
    pm.expect(jsonData).to.have.property("username");
    pm.expect(jsonData).to.have.property("email");
});
```

Exercise 4: POST request

Create data and verify the response

Request

```
POST https://jsonplaceholder.typicode.com/posts

Body:
{
  "title": "TSD test post",
  "body": "Postman API testing exercise",
  "userId": 1
}
```

Tests

```
pm.test("Status code is 201", function () {
  pm.response.to.have.status(201);
});

pm.test("Created post contains title", function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData.title).to.eql("TSD test post");
});
```

Exercise 5: Negative API test

A good API test also verifies error handling

```
GET https://jsonplaceholder.typicode.com/users/999999
```

Expected idea

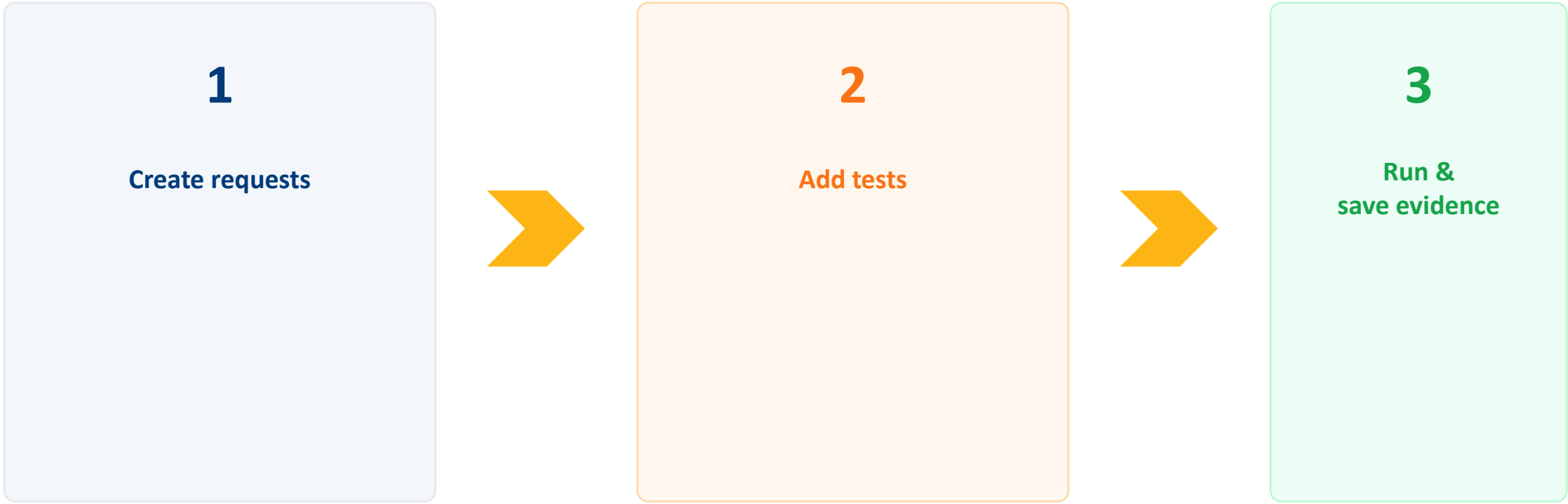
- request invalid or missing resource
- verify error status code
- verify error response if present

```
pm.test("Status code is 404", function () {
    pm.response.to.have.status(404);
});
```

Note: public APIs may behave differently. Document the actual result if it differs.

Run the whole collection

Use Collection Runner to execute the test set



Export runner result screenshot or summary into your report.

Other features

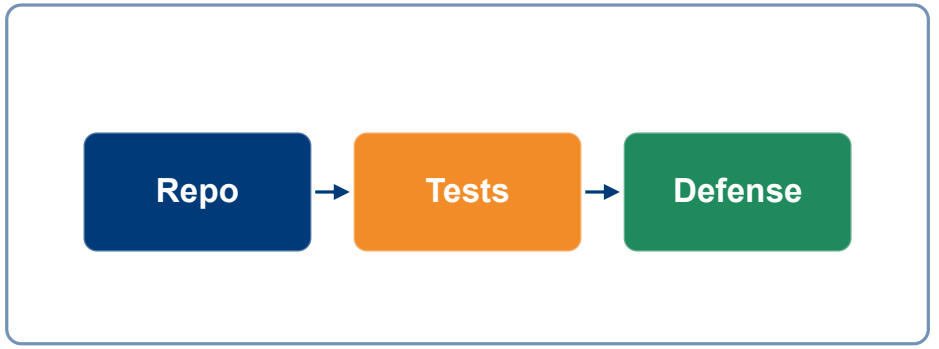
https://www.youtube.com/watch?v=VywxIQ2ZXw4&t=670s

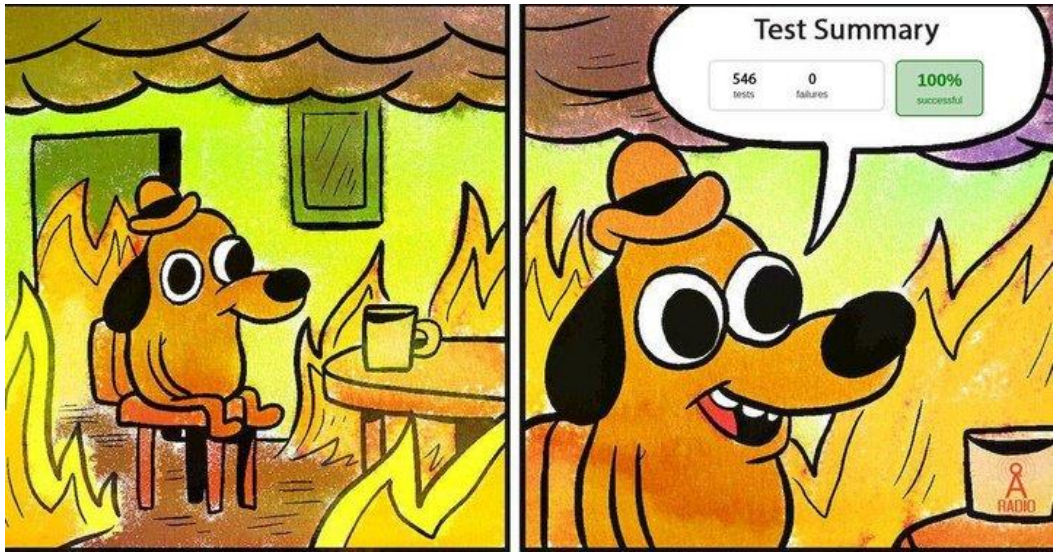
- 31 chapters by functionality

Before the defense

Complete this by the evening of 9 July 2026

1. Finalize repository
2. Export Postman collection
3. Upload final report
4. Upload presentation
5. Prepare your defense
6. Make sure every team member understands their part





Thank you for your attention.

doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk